

RELATÓRIO PARCIAL DE ATIVIDADES

Curso: Bacharelado em Ciência da Computação

Disciplina: Práticas de Extensão I (ajustar, se necessário)

Professor(a): (preencher)

Aluno(a): Rodrigo (ajustar nome completo)

Grupo: (inserir nomes em ordem alfabética, se houver)

Local: Pinhais

Ano: 2026

Subtítulo: Implementação de mudança de tonalidade (pitch shift) no backend do SoundCon usando FFmpeg

1 INTRODUÇÃO

Este relatório parcial apresenta as atividades realizadas na evolução do backend do projeto SoundCon, com foco na adição de uma nova funcionalidade de processamento de áudio: mudança de tonalidade em cents. A proposta foi implementar essa função com robustez técnica, mantendo compatibilidade com o fluxo já existente de conversão de áudio e incluindo mecanismos de proteção da qualidade sonora para o usuário final.

1.1 OBJETIVO

Implementar no backend uma função de pitch shift baseada em FFmpeg, com:

- ? aceitação de parâmetro de transposição em cents;
- ? manutenção da duração do áudio;
- ? classificação de risco de perda de qualidade;
- ? bloqueio de valores extremos;
- ? retorno de metadados para comunicação ao frontend;
- ? testes automatizados de unidade e integração.

2 CONTEXTUALIZAÇÃO

No processamento digital de áudio, alterar tonalidade preservando duração exige algoritmos de time-stretch/pitch-shift. Em ambientes com FFmpeg, essa operação pode ser feita com filtros dedicados (como ``rubberband``) ou por combinação de filtros (``asetrate``, ``aresample``, ``atempo``) quando o filtro dedicado não está disponível.

Durante a análise do ambiente do projeto, verificou-se que o build atual do FFmpeg não possuía `rubberband`. Portanto, a solução adotou estratégia robusta em dois níveis:

1. detectar automaticamente se `rubberband` está disponível;
2. usar fallback de filtros nativos quando necessário.

Também foi definida uma política de limites por cents para equilibrar flexibilidade e qualidade percebida do áudio.

3 DESCRIÇÕES DAS ATIVIDADES REALIZADAS

3.1 MATERIAIS

- ? Node.js (backend existente do projeto);
- ? Express e Multer;
- ? SQLite3;
- ? FFmpeg via `@ffmpeg-installer/ffmpeg`;
- ? Testes com `node --test`;
- ? Documentação oficial FFmpeg (filtros e resampler).

3.2 MÉTODO

As etapas executadas foram:

1. **Levantamento técnico do backend existente**

Identificação do endpoint já existente (`POST /audio/convert`) e do fluxo atual de upload/conversão.

2. **Definição da API sem quebra de compatibilidade**

O endpoint foi mantido e recebeu novo campo opcional `pitchCents`.

3. **Implementação da camada de regras de áudio**

Criação de módulo dedicado com:

- ? parsing e validação de `pitchCents`;
- ? validação de formatos de saída permitidos;
- ? cálculo de razão de pitch ($\text{ratio} = 2^{(\text{cents}/1200)}$);
- ? classificação de qualidade por faixa de cents;

? geração de filter graph conforme engine disponível.

4. ****Engine de processamento com fallback****

? Detecção automática de `rubberband`;

? Se disponível: uso de `rubberband`;

? Se indisponível: `aformat + asetrate + aresample(soxr) + atempo`.

5. ****Robustez operacional****

? troca de `exec` por `spawn` para reduzir risco de injeção;

? timeout por conversão;

? limpeza de arquivos temporários em bloco `finally`;

? logs estruturados em banco com colunas adicionais para auditoria.

6. ****Validação por testes automatizados (TDD)****

? testes unitários para fórmula, validações e classificação;

? testes de integração no endpoint para cenários de sucesso, aviso e bloqueio.

4 RESULTADOS E DISCUSSÕES

4.1 Funcionalidade entregue

A funcionalidade foi implementada com sucesso no backend, com os seguintes comportamentos:

? `pitchCents` opcional (default `0`);

? valores acima de ± 900 cents são bloqueados (`HTTP 422`);

? retorno de metadados:

? `pitch: { cents, ratio, engine }`

? `quality: { level, warning, recommendedRangeCents }`

? preservação do comportamento original quando `pitchCents=0`.

4.2 Política de qualidade aplicada

? `|cents| <= 300`: `safe` (sem aviso);

? `301..600`: `caution` (aviso moderado);

? `601..900`: `high\_risk` (aviso forte);

? ` $>900$ `: bloqueio por alto risco de artefatos.

Essa abordagem orienta o usuário e evita degradações severas de qualidade sonora em transposições extremas.

4.3 Resultados dos testes

Foram executados 9 testes automatizados (unidade + integração), todos aprovados, cobrindo:

- ? cálculo de razão de pitch;
- ? validações de formato e parâmetro;
- ? classificação de risco por faixa de cents;
- ? resposta correta do endpoint para sucesso e bloqueio.

5 CONSIDERAÇÕES FINAIS

O objetivo parcial da atividade foi atendido. A solução implementada é tecnicamente robusta, mantém compatibilidade com o sistema existente e melhora a experiência do usuário ao informar riscos de qualidade de forma explícita.

Como continuidade, recomenda-se:

- ? exibir no frontend os avisos de qualidade retornados pela API;
- ? avaliar atualização do build do FFmpeg para suporte nativo a `rubberband` em produção;
- ? ampliar testes com áudios reais de gêneros diferentes para validação perceptiva.

6 REFERÊNCIAS

- ? FFmpeg. *FFmpeg Filters Documentation*. Disponível em: <https://ffmpeg.org/ffmpeg-filters.html>. Acesso em: 27 maio 2026.
- ? FFmpeg. *FFmpeg Resampler Documentation*. Disponível em: <https://ffmpeg.org/ffmpeg-resampler.html>. Acesso em: 27 maio 2026.
- ? SOUNDCon (repositório local). Arquivos analisados: `backend/server.js`, `backend/audio-processing.js`, `backend/db/db.js`, testes em `backend/test/`.